

TSAR Project Assignment Part 2 - Group 3

Arpad Broglie

Teodor Glisic

2026

Table of contents

Introduction	1
Package installation	1
Task 1: Exploratory Data Analysis	2
Result from DataExplorer::create_report()	3
Task 2: Data Wrangling and Visualization	3
Overall NA per feature	5

Introduction

In this project, we explore the pre-processing step of data wrangling. We do this using dplyr and tidyr. Real-world datasets often are filled with missing values and anomalies. When performing calculations on these dataset this may result in problems. Therefore we first have to perform automated Exploratory Data Analysis (EDA) using the DataExplorer package. This will help to establish an overview of the missing data. Subsequently, we manually wrangle the data to identify and quantify the “dirty” values. We consider empty strings, spaces, NaN values and statistical outliers across all categorical and numerical attributes as dirty values.

The goal is to assess the dataset’s quality to inform future data cleaning decisions.

Package installation

```
if (!require(dplyr)) install.packages("dplyr")
if (!require(tidyr)) install.packages("tidyr")
if (!require(data.table)) install.packages("data.table")
if (!require(DataExplorer)) install.packages("DataExplorer")
if (!require(ggplot2)) install.packages("ggplot2")
```

```
library(dplyr)
library(ggplot2)
library(tidyr)
library(data.table)
library(DataExplorer)
```

Task 1: Exploratory Data Analysis

```
myLCdata <- fread("myLCdata.csv")

dirtyfy <- readRDS(file = "dirtyfy.rds")
myLCdata_dirty <- dirtyfy(myLCdata, 3)

# Executing the DataExplorer::create_report() on the dirtified myLCdata in the console
# create_report(myLCdata_dirty)
```

Result from DataExplorer::create_report()

Missing Data Profile

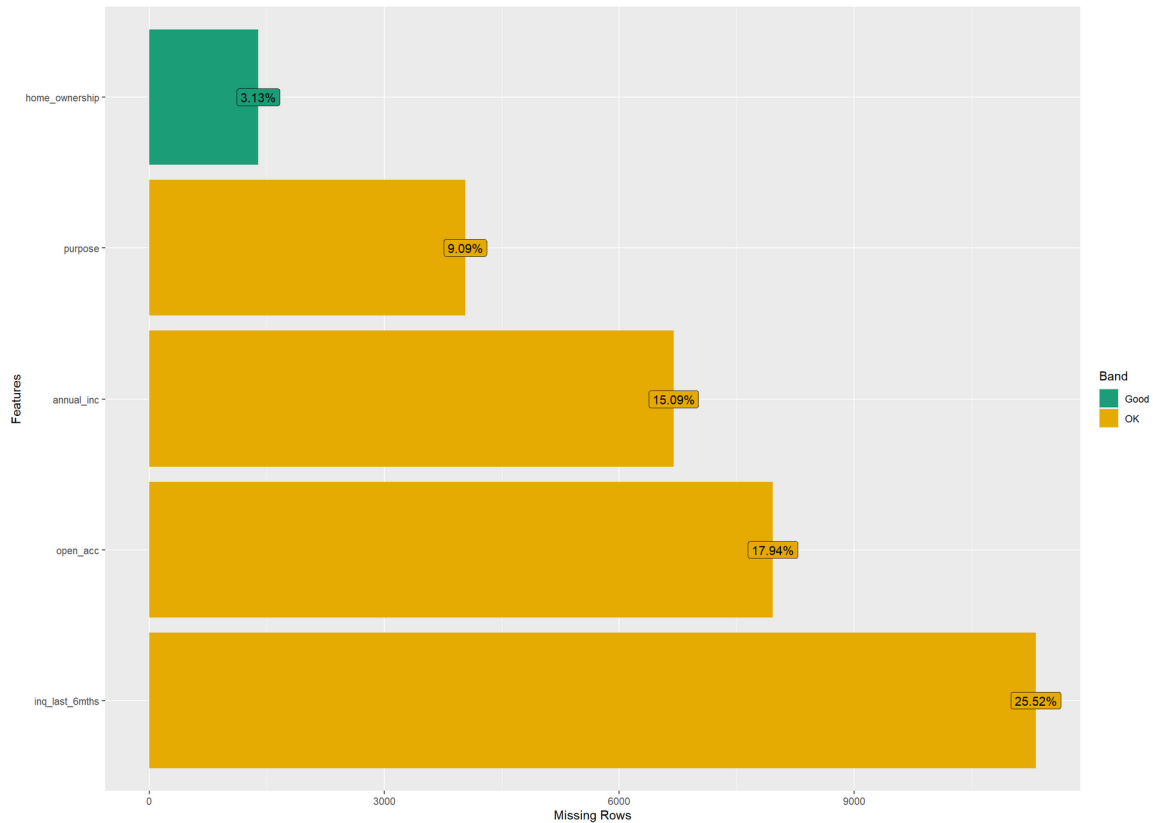


Figure 1: Overview of missing rows by features of the myLCdata_dirty variable

Task 2: Data Wrangling and Visualization

```
glimpse(myLCdata_dirty)
```

Rows: 44,369

Columns: 5

```
$ annual_inc    <dbl> 60000, 18576, NaN, 35000, 125000, 144000, 205000, 11800~  
$ open_acc     <dbl> 6, NaN, 15, 3, NaN, 8, 9, 18, NaN, 3, 11, 10, 15, 5, Na~  
$ purpose      <chr> "credit_card", "", "small_business", "home_improvement"~  
$ home_ownership <chr> "OWN", "RENT", "RENT", "OWN", "RENT", " ", "MORTGAGE", ~  
$ inq_last_6mths <dbl> 0, NA, 2, 1, NA, NA, 0, 0, NA, 0, 0, NA, NA, 0, 0, 0, 1~
```

```
str(myLCdata_dirty)
```

```
Classes 'data.table' and 'data.frame':  44369 obs. of  5 variables:
 $ annual_inc      : num  60000 18576 NaN 35000 125000 ...
 $ open_acc        : num   6 NaN 15 3 NaN 8 9 18 NaN 3 ...
 $ purpose         : chr   "credit_card" "" "small_business" "home_improvement" ...
 $ home_ownership: chr   "OWN" "RENT" "RENT" "OWN" ...
 $ inq_last_6mths: num   0 NA 2 1 NA NaN 0 0 NaN 0 ...
 - attr(*, ".internal.selfref")=<externalptr>
```

```
summary(myLCdata_dirty)
```

annual_inc	open_acc	purpose	home_ownership
Min. : 4000	Min. : 1.00	Length:44369	Length:44369
1st Qu.: 45931	1st Qu.: 8.00	Class :character	Class :character
Median : 65000	Median :11.00	Mode :character	Mode :character
Mean : 75314	Mean :11.58		
3rd Qu.: 90000	3rd Qu.:14.00		
Max. :6000000	Max. :76.00		
NA's :6694	NA's :7962		

inq_last_6mths
Min. : 0.0000
1st Qu.: 0.0000
Median : 0.0000
Mean : 0.6951
3rd Qu.: 1.0000
Max. :31.0000
NA's :11321

Table 1: Explanation of the variables (categorical or numeric)

Variable	Explanation
annual_inc	numerical
open_acc	numerical
purpose	categorical
home_ownership	categorical
inq_last_6mths	numerical

Empty categorical features can contain the following values:

- **NA or Not Available:** Indicates missing values that R recognizes as missing.
- **The string "n/a":** We pretend this string was used in the source data set to indicate missing data. Since it is a string R does not recognize it as missing.
- **A space " ":** Another special string.
- **The empty string "":** Another special string.

Empty numerical features can contain the following values:

- **NA:** Standard missing values.
- **NaN or Not a Number:** Results from invalid computations such as 0/0.
- **Outliers:** These were actually not ingested but already present in the data. There is no rigid mathematical definition of what constitutes an outlier. In this task we use Tukey's fences. We count values as outliers if they lie outside the whiskers of the boxplot.

Overall NA per feature

```
myLCdata_dirty %>% summarise(across(everything(), ~sum(is.na(.))))
```

	annual_inc	open_acc	purpose	home_ownership	inq_last_6mths
1	6694	7962	4033	1390	11321

Categorical features (purpose, home_ownership)

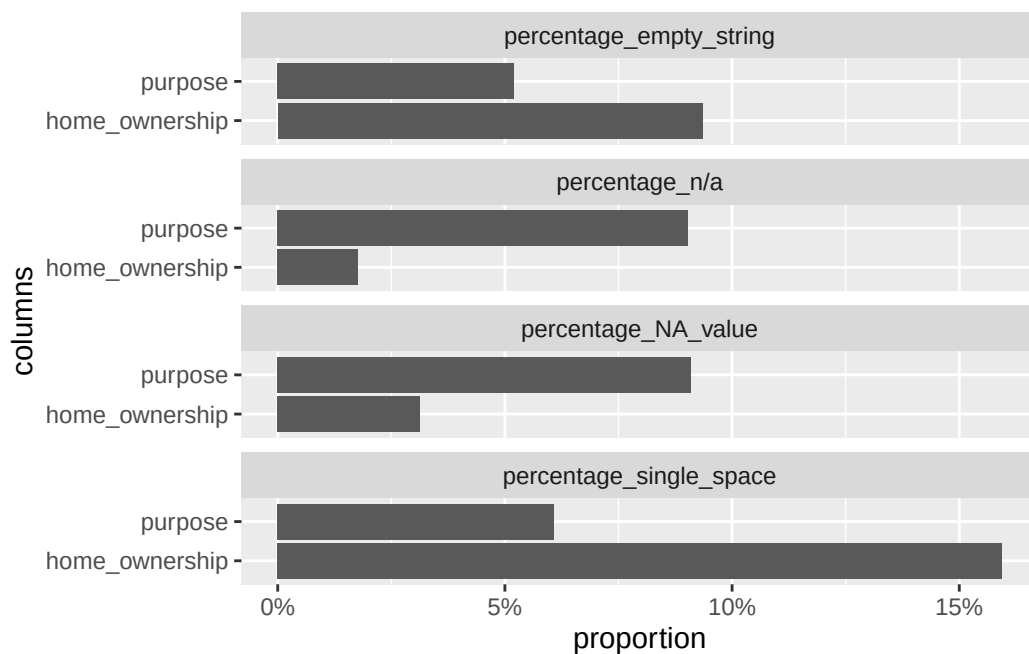
```
myLCdata_dirty_values_categorical <- myLCdata_dirty %>%
  select(purpose, home_ownership) %>%
  pivot_longer(cols = everything(), names_to = "columns", values_to = "empty_value") %>%
  filter(if_any(everything(), ~ . %in% c(" ", "", "n/a") | is.na(.))) %>%
  mutate(empty_value = case_when(
    empty_value == "" ~ "empty_string",
    empty_value == " " ~ "single_space",
    is.na(empty_value) ~ "NA_value",
    empty_value == "n/a" ~ "n/a"
  )) %>%
  group_by(columns, empty_value) %>% summarise(dirty_value_count = n()) %>%
  pivot_wider(names_from = empty_value, values_from=dirty_value_count, names_prefix = "percent")
  mutate(across(where(is.numeric), ~. / nrow(myLCdata_dirty)))

head(myLCdata_dirty_values_categorical)
```

```
# A tibble: 2 x 5
# Groups:   columns [2]
  columns      percentage_NA_value percentage_empty_string `percentage_n/a`
  <chr>          <dbl>          <dbl>          <dbl>
1 home_ownership 0.0313          0.0935          0.0176
2 purpose        0.0909          0.0520          0.0903
# i 1 more variable: percentage_single_space <dbl>
```

```
ggplot_data_categorical <- myLCdata_dirty_values_categorical %>%
  pivot_longer(cols=-columns, names_to = "empty_value", values_to = "proportion")

ggplot(ggplot_data_categorical) +
  geom_col(aes(x = columns, y = proportion)) +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  facet_wrap(~empty_value, ncol = 1)
```



Categorical features 2

Calculate the percentages for all character columns dynamically. As illustrated in @fig-categorical, the character attributes contain various forms of missing data beyond standard NAs.

```

# Calculate percentages for all character columns dynamically
myLCdata_dirty_values_categorical <- myLCdata_dirty %>%
  summarise(
    across(
      where(is.character),
      list(
        perc_NA = ~ sum(is.na(.)) / n(),
        perc_empty_string = ~ sum(. == "", na.rm = TRUE) / n(),
        perc_space = ~ sum(. == " ", na.rm = TRUE) / n(),
        perc_na_string = ~ sum(. == "n/a", na.rm = TRUE) / n()
      )
    )
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("attribute", ".value"),
    names_pattern = "(.*)_(perc_.*)"
  )

# Plotting the categorical data
ggplot_data_categorical <- myLCdata_dirty_values_categorical %>%
  pivot_longer(cols = -attribute, names_to = "empty_value", values_to = "proportion")

ggplot(ggplot_data_categorical) +
  geom_col(aes(x = attribute, y = proportion)) +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  facet_wrap(~empty_value, ncol = 1)

```

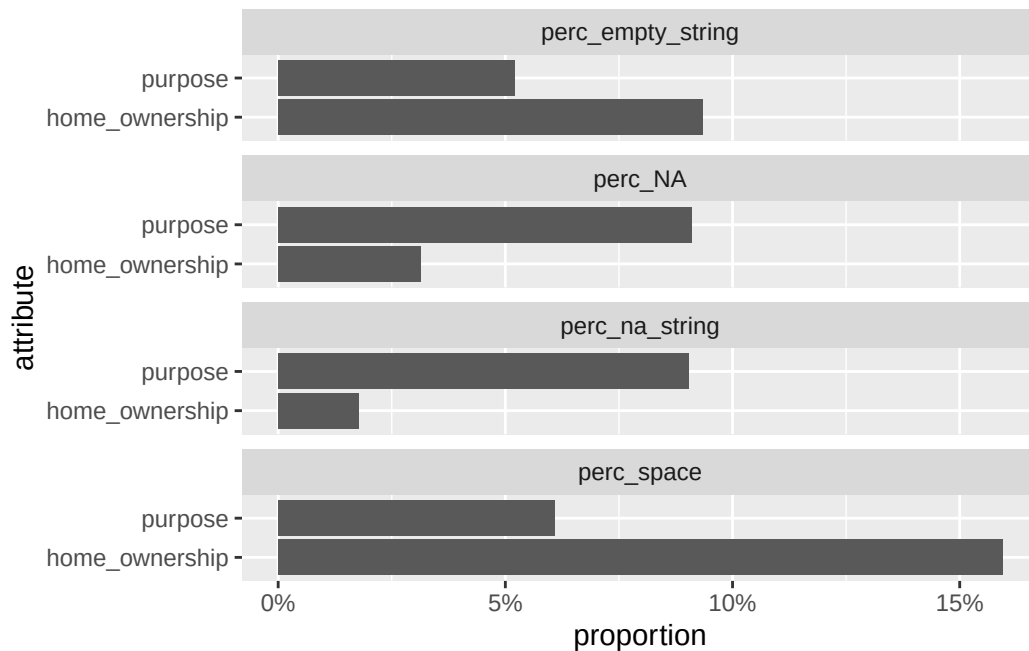


Figure 2: Percentage of special values in the character attributes of myLCdata

Numerical features (annual_inc, open_acc,inq_last_6mths)

```
myLCdata_dirty %>%
  summarise(na_count = sum(is.na(annual_inc)) - sum(is.nan(annual_inc)))
```

```
na_count
1      2192
```

```
myLCdata_dirty %>%
  summarise(NAN_values = sum(is.nan(annual_inc)))
```

```
NAN_values
1      4502
```

```
boxplot.stats(myLCdata_dirty$annual_inc)
```

```
$stats
[1] 4000 45931 65000 90000 156000
```

\$n

[1] 37675

\$conf

[1] 64641.27 65358.73

\$out

[1]	205000.0	184000.0	275000.0	300000.0	275000.0	180000.0	205027.0
[8]	185000.0	160000.0	280000.0	175000.0	182596.0	169000.0	172000.0
[15]	300000.0	160000.0	294000.0	178000.0	225000.0	185000.0	190000.0
[22]	160000.0	200000.0	850000.0	160000.0	220000.0	180000.0	200000.0
[29]	174900.0	270000.0	162500.0	215000.0	170000.0	275000.0	250000.0
[36]	202290.0	255000.0	300000.0	165000.0	204000.0	160000.0	350000.0
[43]	189000.0	165000.0	187834.0	194000.0	200000.0	176200.0	165000.0
[50]	195000.0	165000.0	210000.0	180000.0	200000.0	180000.0	415000.0
[57]	236000.0	192000.0	175000.0	205000.0	302000.0	180000.0	265000.0
[64]	400000.0	159000.0	300000.0	225480.0	188000.0	165000.0	360000.0
[71]	215000.0	240000.0	240000.0	200000.0	170000.0	184000.0	300000.0
[78]	184000.0	190000.0	207000.0	272000.0	194000.0	190000.0	177000.0
[85]	185000.0	200000.0	165000.0	220000.0	158400.0	235000.0	250000.0
[92]	177000.0	225000.0	175000.0	205000.0	175000.0	200000.0	250000.0
[99]	375000.0	185000.0	435000.0	275000.0	210000.0	175000.0	305000.0
[106]	175000.0	314000.0	300000.0	184000.0	165000.0	170692.0	188508.0
[113]	230000.0	160000.0	202000.0	180000.0	340000.0	231000.0	230000.0
[120]	244000.0	250000.0	385000.0	290000.0	160000.0	253000.0	325000.0
[127]	195000.0	160000.0	215099.0	165000.0	180000.0	260000.0	160000.0
[134]	197000.0	158734.0	200000.0	200000.0	200000.0	200000.0	178700.0
[141]	219000.0	200000.0	295000.0	170000.0	225000.0	169800.0	200000.0
[148]	340000.0	168000.0	215000.0	200000.0	220000.0	175000.0	400000.0
[155]	300000.0	170000.0	160000.0	190000.0	194000.0	230000.0	185000.0
[162]	163000.0	185000.0	175000.0	200000.0	162000.0	225000.0	180000.0
[169]	300000.0	225000.0	1200000.0	225000.0	167000.0	165000.0	252000.0
[176]	355000.0	232000.0	300000.0	268000.0	181752.0	165000.0	175000.0
[183]	175000.0	157000.0	163000.0	200000.0	160000.0	280000.0	160000.0
[190]	165000.0	170000.0	187496.0	180000.0	300000.0	190000.0	300000.0
[197]	180000.0	219000.0	225000.0	175000.0	160000.0	350000.0	175000.0
[204]	365000.0	162000.0	170000.0	1500000.0	270000.0	175000.0	225000.0
[211]	170000.0	160000.0	250000.0	200000.0	160000.0	275000.0	250000.0
[218]	175000.0	300009.0	311674.0	300000.0	181000.0	240000.0	190000.0
[225]	271000.0	185000.0	200000.0	160000.0	190000.0	225000.0	191000.0
[232]	240000.0	240000.0	160000.0	170000.0	200000.0	320000.0	240000.0
[239]	180000.0	180000.0	178000.0	172000.0	360000.0	165000.0	160000.0

[246]	163000.0	300000.0	208000.0	220000.0	175000.0	184000.0	200000.0
[253]	165000.0	158000.0	173000.0	168000.0	200000.0	160000.0	180000.0
[260]	170000.0	425000.0	200000.0	175000.0	249000.0	236540.0	800000.0
[267]	162000.0	160000.0	190000.0	225000.0	172000.0	185000.0	160000.0
[274]	200000.0	206000.0	350000.0	200000.0	165000.0	165000.0	200000.0
[281]	250000.0	165000.0	161000.0	160000.0	190329.0	209000.0	205000.0
[288]	260000.0	220000.0	203000.0	200000.0	200529.0	188000.0	181596.0
[295]	350000.0	160000.0	166000.0	165000.0	159000.0	198000.0	180000.0
[302]	165000.0	175000.0	275000.0	180000.0	250000.0	278000.0	220000.0
[309]	344000.0	200000.0	200000.0	192000.0	225000.0	225000.0	200000.0
[316]	160000.0	200000.0	180000.0	264000.0	165000.0	330000.0	175656.0
[323]	191500.0	400000.0	175000.0	333000.0	384000.0	225000.0	183000.0
[330]	173000.0	199000.0	180000.0	365000.0	200000.0	2000000.0	165000.0
[337]	600000.0	205215.0	180000.0	930000.0	160000.0	160000.0	190000.0
[344]	160000.0	200000.0	170000.0	169000.0	225000.0	275000.0	230000.0
[351]	220000.0	200000.0	174000.0	185500.0	160000.0	200000.0	250000.0
[358]	350000.0	200000.0	215000.0	165000.0	183000.0	180000.0	160000.0
[365]	175000.0	227000.0	430000.0	170000.0	200000.0	157500.0	175000.0
[372]	186500.0	195000.0	216000.0	170000.0	180000.0	240000.0	166000.0
[379]	250000.0	250000.0	180000.0	300000.0	465580.0	201000.0	163000.0
[386]	166000.0	210000.0	300000.0	197540.0	215000.0	160000.0	172000.0
[393]	350000.0	201000.0	165000.0	198000.0	250000.0	220000.0	170000.0
[400]	400000.0	166000.0	350000.0	250000.0	193000.0	180000.0	160000.0
[407]	170000.0	169000.0	275000.0	176000.0	185000.0	255000.0	160000.0
[414]	450000.0	180000.0	170000.0	225000.0	200000.0	215000.0	175000.0
[421]	350000.0	235000.0	300000.0	250000.0	200000.0	240000.0	165000.0
[428]	160000.0	177000.0	198000.0	225000.0	200000.0	165000.0	180000.0
[435]	240000.0	493433.0	295000.0	161567.0	250000.0	300000.0	175000.0
[442]	180000.0	210000.0	165000.0	188000.0	180000.0	170000.0	180000.0
[449]	200000.0	178000.0	250000.0	265000.0	380000.0	161000.0	225500.0
[456]	272000.0	180000.0	425000.0	160000.0	300000.0	185000.0	225000.0
[463]	160000.0	196000.0	170000.0	400000.0	221000.0	187400.0	225000.0
[470]	300000.0	175000.0	176000.0	179000.0	175000.0	170000.0	200000.0
[477]	180000.0	165000.0	750000.0	165000.0	180000.0	225000.0	160000.0
[484]	250000.0	168750.0	200000.0	233000.0	225000.0	162000.0	265000.0
[491]	165000.0	200000.0	198000.0	200000.0	190912.0	303000.0	285000.0
[498]	157800.0	168000.0	300000.0	550000.0	170000.0	225000.0	220000.0
[505]	164000.0	215000.0	350000.0	190000.0	158000.0	190000.0	160000.0
[512]	170000.0	160000.0	206500.0	250000.0	171000.0	220000.0	300000.0
[519]	170000.0	225000.0	245000.0	175000.0	182000.0	210000.0	320000.0
[526]	200000.0	175000.0	225000.0	169000.0	225000.0	300000.0	200000.0
[533]	223000.0	165000.0	170000.0	175000.0	208000.0	225000.0	215000.0
[540]	200000.0	160000.0	185000.0	285000.0	202000.0	185000.0	307000.0

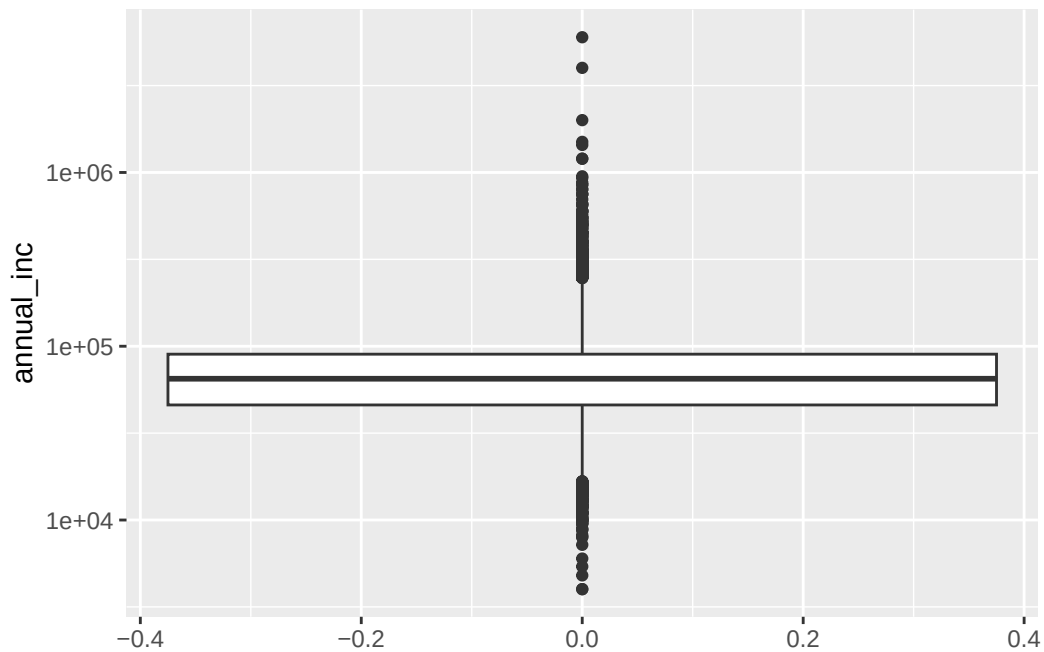
[547]	170000.0	175000.0	182000.0	160000.0	245000.0	175000.0	188709.0
[554]	170000.0	250000.0	165000.0	165301.0	325000.0	164000.0	170000.0
[561]	250000.0	189000.0	231000.0	177500.0	300000.0	200000.0	167000.0
[568]	300000.0	246000.0	200000.0	200000.0	186000.0	165000.0	170000.0
[575]	540000.0	200000.0	220000.0	192000.0	170000.0	250000.0	250000.0
[582]	235000.0	190000.0	200000.0	160000.0	175000.0	220000.0	214704.0
[589]	215000.0	525000.0	157000.0	197000.0	235000.0	350000.0	200000.0
[596]	450000.0	184000.0	174000.0	175000.0	190000.0	160000.0	185000.0
[603]	207182.0	250000.0	160000.0	243000.0	230000.0	317000.0	350000.0
[610]	163000.0	157000.0	200000.0	165000.0	250000.0	170000.0	170000.0
[617]	170000.0	350000.0	183000.0	175000.0	185000.0	200000.0	220000.0
[624]	212000.0	285000.0	260000.0	160000.0	350000.0	275000.0	200000.0
[631]	175000.0	180000.0	260000.0	200000.0	200000.0	162000.0	265000.0
[638]	375000.0	250000.0	176000.0	200000.0	315000.0	375000.0	249000.0
[645]	184000.0	160000.0	170000.0	160000.0	168000.0	180000.0	350000.0
[652]	450000.0	180000.0	180000.0	1440000.0	195000.0	390000.0	235000.0
[659]	480000.0	250000.0	210000.0	165500.0	274992.0	165000.0	243000.0
[666]	162000.0	237500.0	175000.0	260000.0	500000.0	182224.4	250000.0
[673]	170000.0	220000.0	220000.0	230000.0	168000.0	300000.0	175000.0
[680]	282000.0	175000.0	160000.0	260000.0	210000.0	200000.0	170000.0
[687]	250000.0	175000.0	165000.0	317000.0	275000.0	250000.0	250000.0
[694]	160000.0	165000.0	225000.0	170000.0	700000.0	300000.0	177800.0
[701]	168000.0	165000.0	190000.0	220000.0	160000.0	160000.0	200000.0
[708]	160278.0	330000.0	175000.0	200000.0	240000.0	160000.0	200000.0
[715]	180000.0	325000.0	246720.0	160000.0	195000.0	195000.0	660000.0
[722]	200000.0	265000.0	200000.0	180000.0	165000.0	1200000.0	195000.0
[729]	160000.0	162437.0	648000.0	400000.0	165000.0	350000.0	194000.0
[736]	170000.0	180000.0	167800.0	180000.0	162000.0	200000.0	160000.0
[743]	175000.0	170000.0	225000.0	200000.0	225000.0	178000.0	160000.0
[750]	250000.0	160000.0	210000.0	160000.0	350000.0	190000.0	450000.0
[757]	250000.0	237000.0	160000.0	180000.0	222000.0	195000.0	200000.0
[764]	200000.0	200000.0	180000.0	240000.0	223000.0	160000.0	160000.0
[771]	180000.0	200000.0	200000.0	170000.0	178000.0	165000.0	200000.0
[778]	160000.0	195000.0	170000.0	216000.0	225000.0	188000.0	180000.0
[785]	255000.0	260000.0	325000.0	211680.0	167000.0	160000.0	180000.0
[792]	168000.0	185000.0	450000.0	274000.0	170000.0	200000.0	160000.0
[799]	200000.0	225000.0	160000.0	160000.0	200000.0	185000.0	197600.0
[806]	180000.0	600000.0	174000.0	225000.0	170000.0	188000.0	231000.0
[813]	277000.0	200000.0	170000.0	160000.0	156800.0	400000.0	160000.0
[820]	165000.0	218000.0	219000.0	180000.0	180000.0	200000.0	240000.0
[827]	170000.0	160000.0	450000.0	241000.0	200000.0	290000.0	171250.0
[834]	200000.0	225000.0	165000.0	190000.0	159000.0	225000.0	200000.0
[841]	175000.0	160000.0	220000.0	175000.0	183000.0	240000.0	170000.0

[848]	200000.0	208000.0	210000.0	160000.0	200000.0	161000.0	250000.0
[855]	200000.0	190000.0	225000.0	200000.0	182000.0	190000.0	261000.0
[862]	160000.0	220000.0	187000.0	240384.0	215000.0	160000.0	172300.0
[869]	170000.0	165000.0	198000.0	190000.0	315000.0	280000.0	190000.0
[876]	480000.0	230000.0	175000.0	160000.0	210000.0	185000.0	185000.0
[883]	250000.0	250000.0	170000.0	176000.0	160000.0	4000095.0	240000.0
[890]	234000.0	175000.0	435000.0	237000.0	160000.0	200000.0	210000.0
[897]	200000.0	170000.0	240000.0	205000.0	224000.0	295500.0	175000.0
[904]	165000.0	162000.0	197000.0	180000.0	198000.0	225000.0	175000.0
[911]	160000.0	175000.0	167000.0	195000.0	450000.0	175000.0	193000.0
[918]	200000.0	185000.0	184660.0	165000.0	160000.0	165000.0	180000.0
[925]	180000.0	160000.0	180000.0	163000.0	250000.0	230000.0	315000.0
[932]	160000.0	190000.0	160000.0	200000.0	235000.0	200000.0	280000.0
[939]	202888.0	177000.0	275000.0	172000.0	180000.0	210000.0	205000.0
[946]	225000.0	161000.0	200000.0	170000.0	255000.0	205000.0	198000.0
[953]	400000.0	205000.0	168000.0	170000.0	176000.0	297000.0	206000.0
[960]	240000.0	163000.0	210000.0	190000.0	200000.0	165000.0	160000.0
[967]	157000.0	160000.0	300000.0	175000.0	185000.0	208000.0	210000.0
[974]	168480.0	180000.0	250000.0	235000.0	165000.0	350000.0	160000.0
[981]	194100.0	344000.0	750000.0	176000.0	250000.0	174000.0	210000.0
[988]	170000.0	250000.0	175000.0	170000.0	200000.0	225000.0	160000.0
[995]	180000.0	200000.0	200000.0	310000.0	525000.0	300000.0	185000.0
[1002]	175000.0	250000.0	178008.0	360000.0	193000.0	165000.0	250000.0
[1009]	300000.0	160000.0	340000.0	210000.0	165000.0	218000.0	200000.0
[1016]	200000.0	400000.0	355000.0	200000.0	165000.0	247000.0	160000.0
[1023]	260000.0	160000.0	200000.0	217000.0	250000.0	158000.0	245000.0
[1030]	163000.0	180000.0	195000.0	165000.0	200000.0	230000.0	165000.0
[1037]	200000.0	285000.0	515000.0	160000.0	191000.0	235000.0	220000.0
[1044]	175000.0	190000.0	225000.0	185000.0	325000.0	260000.0	170000.0
[1051]	160000.0	175000.0	235000.0	205000.0	180000.0	160000.0	245000.0
[1058]	200000.0	500000.0	170000.0	161000.0	174000.0	180000.0	357000.0
[1065]	300000.0	165000.0	205000.0	280000.0	160000.0	175000.0	220000.0
[1072]	160000.0	240000.0	200000.0	225000.0	170000.0	190000.0	292000.0
[1079]	270000.0	525000.0	400000.0	165000.0	185000.0	167000.0	300000.0
[1086]	175000.0	182000.0	187000.0	250000.0	342500.0	230000.0	165000.0
[1093]	247000.0	250000.0	500000.0	175000.0	200000.0	200000.0	208000.0
[1100]	180000.0	220000.0	160000.0	215000.0	237000.0	250000.0	320000.0
[1107]	200000.0	160000.0	175000.0	220000.0	225000.0	360000.0	229933.0
[1114]	162000.0	205000.0	189000.0	215000.0	165000.0	160000.0	550000.0
[1121]	212000.0	176500.0	185000.0	180000.0	165000.0	170000.0	200000.0
[1128]	213000.0	250000.0	185000.0	160000.0	259000.0	175000.0	178000.0
[1135]	200000.0	162000.0	198000.0	184000.0	202000.0	234000.0	200000.0
[1142]	159000.0	230153.8	220000.0	174000.0	181000.0	200000.0	200000.0

[1149]	170000.0	165000.0	500000.0	176000.0	260000.0	160000.0	161595.0
[1156]	200000.0	410000.0	160000.0	175000.0	260000.0	600000.0	330000.0
[1163]	235000.0	191000.0	165000.0	302000.0	200000.0	160000.0	179000.0
[1170]	250000.0	200000.0	185000.0	180000.0	180000.0	170000.0	180000.0
[1177]	157500.0	167000.0	165000.0	181000.0	350000.0	185000.0	650000.0
[1184]	250000.0	170000.0	200000.0	168000.0	220000.0	300000.0	250000.0
[1191]	210000.0	160000.0	160000.0	160000.0	160000.0	190000.0	252946.9
[1198]	275000.0	215033.0	250000.0	240000.0	165000.0	200000.0	207000.0
[1205]	180000.0	175000.0	225000.0	190000.0	160000.0	182000.0	160000.0
[1212]	300000.0	350000.0	225000.0	165000.0	188000.0	165000.0	165000.0
[1219]	320000.0	190000.0	180000.0	200000.0	180000.0	180000.0	196000.0
[1226]	325000.0	195000.0	172000.0	200000.0	275000.0	506350.0	275000.0
[1233]	205000.0	230000.0	160000.0	160000.0	165000.0	175000.0	250000.0
[1240]	350000.0	215000.0	170000.0	160000.0	200000.0	165000.0	200000.0
[1247]	165000.0	215000.0	165000.0	520000.0	250000.0	200000.0	165000.0
[1254]	170000.0	375000.0	185000.0	210000.0	312858.4	200000.0	280000.0
[1261]	165000.0	175000.0	195000.0	170000.0	200000.0	157500.0	184000.0
[1268]	160000.0	300000.0	193000.0	230000.0	565000.0	275000.0	170448.0
[1275]	158000.0	875000.0	160000.0	180952.0	194000.0	187500.0	300000.0
[1282]	170000.0	205000.0	160000.0	450000.0	166000.0	210000.0	175000.0
[1289]	160000.0	191000.0	167400.0	450000.0	380000.0	170000.0	180000.0
[1296]	176000.0	160000.0	350000.0	216000.0	262540.0	175000.0	280000.0
[1303]	450000.0	360000.0	185000.0	200000.0	300000.0	240000.0	225000.0
[1310]	398000.0	185000.0	180000.0	206000.0	288000.0	160000.0	225000.0
[1317]	160000.0	268000.0	250000.0	200000.0	157000.0	228000.0	213000.0
[1324]	175000.0	300000.0	195000.0	176500.0	350000.0	220000.0	180000.0
[1331]	160000.0	185000.0	239000.0	300000.0	170000.0	217000.0	160000.0
[1338]	165000.0	185738.2	198000.0	340000.0	279000.0	168000.0	320000.0
[1345]	290260.0	180000.0	180000.0	168000.0	180000.0	230000.0	211000.0
[1352]	215000.0	384000.0	260000.0	275000.0	750000.0	202000.0	185000.0
[1359]	180000.0	190000.0	200000.0	400000.0	175000.0	180000.0	340000.0
[1366]	295000.0	168000.0	165000.0	180000.0	186000.0	170000.0	197000.0
[1373]	214000.0	245000.0	200000.0	160000.0	210000.0	300000.0	176000.0
[1380]	800000.0	162000.0	220640.0	180000.0	220000.0	185000.0	185000.0
[1387]	160000.0	400000.0	290000.0	160000.0	250000.0	170000.0	165000.0
[1394]	200000.0	200000.0	195000.0	165000.0	265000.0	178000.0	181251.0
[1401]	210000.0	342000.0	215000.0	210000.0	240000.0	350000.0	160000.0
[1408]	250000.0	170000.0	185000.0	250142.0	185000.0	217900.0	165000.0
[1415]	225000.0	273000.0	245000.0	200000.0	160000.0	165000.0	220000.0
[1422]	157000.0	190000.0	350000.0	170000.0	225000.0	336000.0	240000.0
[1429]	250000.0	170000.0	270000.0	175000.0	220000.0	202000.0	250000.0
[1436]	160000.0	300000.0	180000.0	160000.0	160000.0	204000.0	300000.0
[1443]	400000.0	203000.0	168000.0	227000.0	160000.0	275000.0	160000.0

[1450]	160000.0	160000.0	200000.0	240000.0	250000.0	225000.0	175000.0
[1457]	212000.0	195000.0	225000.0	163000.0	190000.0	180000.0	175000.0
[1464]	174000.0	178000.0	550000.0	158000.0	280000.0	400000.0	160000.0
[1471]	350000.0	6000000.0	168000.0	170000.0	169500.0	210000.0	240000.0
[1478]	162000.0	172908.0	185000.0	200000.0	183000.0	201000.0	200000.0
[1485]	165000.0	230000.0	600000.0	200000.0	168000.0	292000.0	180000.0
[1492]	165000.0	547000.0	165000.0	159000.0	200000.0	196000.0	170000.0
[1499]	201000.0	225000.0	200000.0	200000.0	160000.0	225000.0	158000.0
[1506]	585000.0	190000.0	205000.0	185000.0	210000.0	206200.0	225000.0
[1513]	235000.0	175000.0	168000.0	170000.0	160000.0	170000.0	160000.0
[1520]	160000.0	170000.0	169400.0	180000.0	235000.0	168000.0	180000.0
[1527]	170000.0	182000.0	250000.0	212000.0	175000.0	190000.0	160000.0
[1534]	160000.0	204000.0	180000.0	190000.0	160000.0	165000.0	182000.0
[1541]	165000.0	165000.0	275000.0	175000.0	300000.0	210000.0	178000.0
[1548]	205000.0	240000.0	200000.0	195000.0	162000.0	185000.0	170000.0
[1555]	250000.0	185000.0	300000.0	200000.0	225000.0	185000.0	190000.0
[1562]	200000.0	175000.0	400000.0	275000.0	200000.0	206000.0	270080.0
[1569]	180000.0	240000.0	194000.0	165000.0	190000.0	165000.0	200000.0
[1576]	180000.0	172000.0	300000.0	174000.0	425000.0	350000.0	200000.0
[1583]	169000.0	175000.0	216000.0	225000.0	170000.0	157000.0	250000.0
[1590]	300000.0	170000.0	271000.0	170000.0	222000.0	185000.0	180000.0
[1597]	500000.0	160000.0	260000.0	200000.0	160000.0	160000.0	180000.0
[1604]	160000.0	270000.0	175000.0	170000.0	180000.0	180000.0	200000.0
[1611]	160000.0	310000.0	195000.0	185000.0	175000.0	160000.0	275000.0
[1618]	165000.0	171258.0	160000.0	175000.0	850000.0	171600.0	200000.0
[1625]	190000.0	186000.0	175000.0	240000.0	200000.0	170000.0	950000.0
[1632]	160000.0	300000.0	350000.0	170000.0	235200.0	205000.0	162000.0
[1639]	175000.0	315000.0	160000.0	170000.0	216000.0	188000.0	300000.0
[1646]	160000.0	160000.0	400000.0	170000.0	175000.0	204218.0	190200.0
[1653]	165000.0	180000.0	275000.0	250000.0	325000.0	160000.0	262000.0
[1660]	300000.0	265000.0	200000.0	185000.0	210000.0	300000.0	160000.0
[1667]	211000.0	183000.0	200000.0	200000.0	305000.0	240000.0	168000.0
[1674]	175000.0	201000.0	212000.0	160000.0	337871.0	195000.0	215000.0
[1681]	200000.0	195000.0	256000.0	350000.0	205000.0	342500.0	400000.0
[1688]	200000.0	200000.0	220000.0	226000.0	250000.0	190000.0	270000.0
[1695]	200000.0	180000.0	158700.0	475000.0	205000.0	250000.0	170000.0
[1702]	168000.0	240000.0	175000.0	221000.0	277343.0	180000.0	160000.0
[1709]	240000.0	250000.0	165000.0	248000.0	438000.0	170000.0	210000.0
[1716]	185000.0	180000.0	200000.0	160000.0	250000.0	195000.0	239000.0
[1723]	504000.0	170000.0	280000.0	160000.0	200000.0	190000.0	660000.0

```
ggplot(myLCdata_dirty) + geom_boxplot(aes(y=annual_inc)) + scale_y_log10()
```



```
# all rows
nrow(myLCdata_dirty)
```

```
[1] 44369
```

```
# is.na() returns TRUE for both NA and NaN, however is.na() return TRUE for NaN (0/0) and I
myLCdata_dirty %>% select(annual_inc, open_acc, inq_last_6mths) %>% filter(if_any(everything
```

```
annual_inc open_acc inq_last_6mths
1      6694      7962      11321
```

```
# just NA values
myLCdata_dirty %>%
  select(annual_inc, open_acc, inq_last_6mths) %>%
  filter(if_any(everything(), ~is.na(.))) %>%
  summarise(across(everything(), ~sum(is.na(.))-sum(is.nan(.))))
```

```
annual_inc open_acc inq_last_6mths
1      2192      3338      4081
```

```
# just NAN values
myLCdata_dirty %>%
  select(annual_inc, open_acc, inq_last_6mths) %>%
  filter(if_any(everything(), ~is.na(.))) %>%
  summarise(across(everything(), ~sum(is.nan(.))))
```

	annual_inc	open_acc	inq_last_6mths
1	4502	4624	7240

```
# outliers in features
myLCdata_dirty %>%
  select(annual_inc, open_acc, inq_last_6mths) %>%
  summarise(across(everything(), ~length(boxplot.stats(.x)$out)))
```

	annual_inc	open_acc	inq_last_6mths
1	1729	1145	2022

```
# proportional table of NA and NAN values
table_NA_NAN_perc <- myLCdata_dirty %>%
  select(annual_inc, open_acc, inq_last_6mths) %>%
  pivot_longer(cols = everything(), names_to = "feature", values_to = "values") %>%
  filter(if_any(everything(), ~is.na(.))) %>%
  group_by(feature, values) %>%
  summarise(count = n()) %>%
  mutate(across(is.numeric, ~./nrow(myLCdata_dirty))) %>%
  pivot_wider(names_from = values, values_from = count, names_prefix = "percentage_")
```

```
# length of outliers
table_outliers_perc <- myLCdata_dirty %>%
  select(annual_inc, open_acc, inq_last_6mths) %>%
  summarise(across(everything(), ~length(boxplot.stats(.x)$out))) %>%
  pivot_longer(cols = everything(), names_to = "feature", values_to = "percentage_outliers")
  mutate(across(is.numeric, ~./nrow(myLCdata_dirty)))
```

```
#Joining the two tables together (I understand it could be done quicker, but I wanted to test)
analysis_missing_numeric <- table_NA_NAN_perc %>% inner_join(table_outliers_perc, by="feature")
analysis_missing_numeric
```

```
# A tibble: 3 x 4
```

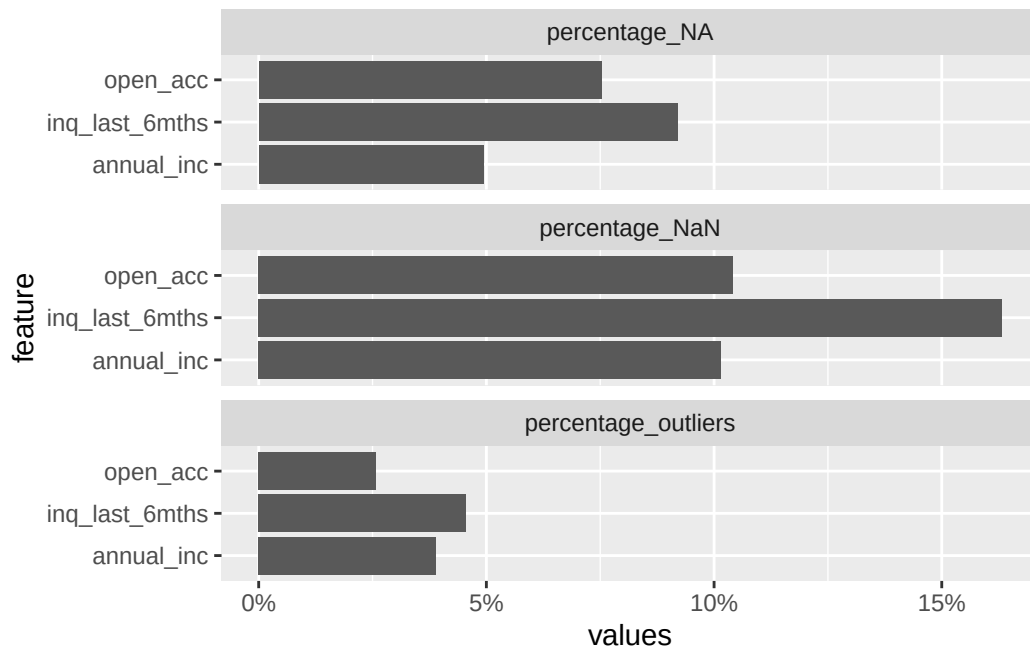
```
# Groups:   feature [3]
  feature      percentage_NaN percentage_NA percentage_outliers
  <chr>          <dbl>          <dbl>          <dbl>
1 annual_inc      0.101          0.0494         0.0390
2 inq_last_6mths  0.163          0.0920         0.0456
3 open_acc        0.104          0.0752         0.0258
```

```
#The same just shorter (from Gemini for reference)
myLCdata_dirty %>%
  summarise(
    across(
      c(annual_inc, open_acc, inq_last_6mths),
      list(
        percentage_NaN = ~ sum(is.nan(.x)) / n(),
        percentage_NA = ~ sum(is.na(.x) & !is.nan(.x)) / n(),
        percentage_outliers = ~ length(boxplot.stats(.x)$out) / n()
      )
    )
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("feature", ".value"),
    names_pattern = "(.*)_(percentage_.*)"
  )
```

```
# A tibble: 3 x 4
  feature      percentage_NaN percentage_NA percentage_outliers
  <chr>          <dbl>          <dbl>          <dbl>
1 annual_inc      0.101          0.0494         0.0390
2 open_acc        0.104          0.0752         0.0258
3 inq_last_6mths  0.163          0.0920         0.0456
```

```
ggplot_data_numeric <- analysis_missing_numeric %>%
  pivot_longer(-feature, names_to = "type", values_to = "values")

ggplot(ggplot_data_numeric) +
  geom_col(aes(x=feature, y=values)) +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  facet_wrap(~type, ncol = 1)
```



Numerical Features 2

@fig-numerical details the prevalence of standard missing values, invalid computations (NaN), and outliers across all numerical attributes.

```
# Calculate percentages for all numerical columns dynamically
analysis_missing_numeric <- myLCdata_dirty %>%
  summarise(
    across(
      where(is.numeric),
      list(
        perc_NA = ~ sum(is.na(.) & !is.nan(.)) / n(),
        perc_NaN = ~ sum(is.nan(.)) / n(),
        perc_outliers = ~ length(boxplot.stats(.)$out) / n()
      )
    )
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("attribute", ".value"),
    names_pattern = "(.*)_(perc_.*)"
  )

head(analysis_missing_numeric)
```

```
# A tibble: 3 x 4
  attribute      perc_NA perc_NaN perc_outliers
  <chr>          <dbl>   <dbl>         <dbl>
1 annual_inc    0.0494   0.101         0.0390
2 open_acc      0.0752   0.104         0.0258
3 inq_last_6mths 0.0920   0.163         0.0456

# Plotting the numerical data
ggplot_data_numeric <- analysis_missing_numeric %>%
  pivot_longer(cols = -attribute, names_to = "type", values_to = "proportion")

ggplot(ggplot_data_numeric) +
  geom_col(aes(x = attribute, y = proportion)) +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  facet_wrap(~type, ncol = 1)
```

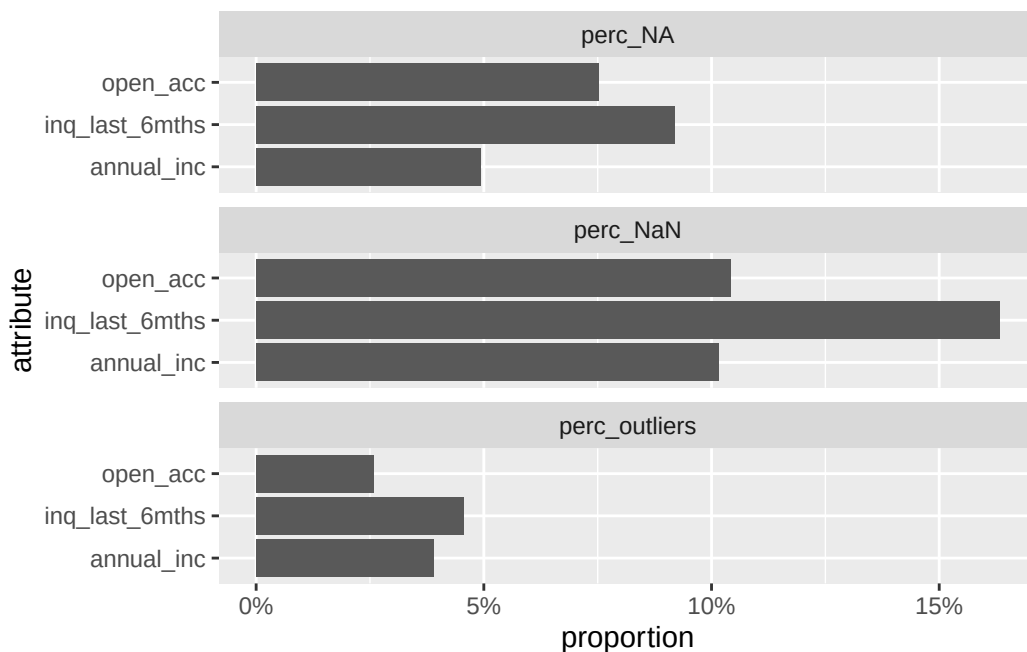


Figure 3: Percentage of special values in the numerical attributes of myLCdata

Conclusion

Through this analysis we found the missing and special values within our dataset. While automated tools like DataExplorer are excellent for identifying standard NA values, our manual

wrangling revealed that significant portions of the missing data we disguised as empty string, spaces, “n/a” text strings.

Using Tukey’s fences allowed us to systematically isolate outliers in numerical features. By combining `dplyr` for calculation and `tidyr` for reshaping the data, we were able to effectively visualize the true extent of the dataset’s inconsistencies. This laid the groundwork for the formal data cleaning phase.